

The Ouroboros Engine: A Bare-Metal Cryptographic Authentication Framework in Pure AVR Assembly

Kevin Thomas

School of Computing and Information
University of Pittsburgh, Pittsburgh, PA, USA
 ket189@pitt.edu

Abstract — We present the Ouroboros Engine, a cryptographic authentication system implemented entirely in hand-crafted AVR assembly language for the Atmel ATmega328P microcontroller operating at 8 MHz without an external crystal. The system accepts a user passphrase over a 9600-baud serial interface, subjects it to a key-stretching Davies-Meyer hash construction executing 24,576 sequential iterations of Speck-128/256 — consuming approximately 11 seconds of real wall-clock time on the target hardware — derives a CTR-mode keystream, and decrypts an encrypted bytecode payload from program flash. Integrity is verified with a fixed-iteration branchless MAC check. On success, a minimal 4-opcode bytecode interpreter drives WS2812B RGB LEDs and transmits a response string over UART. The design incorporates hardware-timer-derived timing jitter, constant-iteration cipher scanning, and post-use key material zeroization. The entire system — cipher, key schedule, hash construction, CTR engine, MAC verifier, bytecode dispatcher, LED driver, UART driver, and state machine — is implemented in approximately 550 lines of commented assembly with no C runtime, no operating system, no external libraries, and no interrupt handlers.

Index Terms — AVR assembly, Speck cipher, Davies-Meyer hash, CTR mode, WS2812B, embedded cryptography, implementation analysis, bare-metal.

I. Introduction

Embedded cryptographic systems are traditionally implemented in C or C++, relying on established library ecosystems (e.g., wolfSSL, mbed TLS, TinyCrypt) that abstract the underlying hardware. While pragmatic, this approach obscures the relationship between the algorithm specification and the hardware execution model, introduces dependencies on compiler behaviour, and can preclude fine-grained cycle-level implementation analysis.

This paper describes the **Ouroboros Engine**, a self-contained authentication system written exclusively in AVR assembly (AT&T syntax, assembled with `avr-as`) targeting the ATmega328P — a ubiquitous 8-bit microcontroller with 32 KB flash, 2 KB SRAM, and 1 KB EEPROM. The design uses no C runtime (`avr-libc`), no interrupt service routines, and no hardware peripherals beyond UART, GPIO, and the free-running

Timer/Counter 0 register (used as an entropy source for jitter).

The central cryptographic primitive is Speck-128/256, an ARX (Add-Rotate-XOR) cipher designed by the NSA in 2013 specifically for constrained hardware [1]. On an 8-bit AVR, the 64-bit word operations of Speck must be decomposed into byte-level register manipulations — we describe this decomposition in full and verify its equivalence to the reference specification via a Python golden model included with the source.

The system is named “Ouroboros” after the ancient symbol of a serpent devouring its own tail: the hash output that serves as the CTR nonce was derived by encrypting the hash state with the same key used to decrypt the payload — a circular, self-referential cryptographic dependency in which the key both creates and unlocks the cipher.

II. Related Work

Speck and Simon were designed for constrained environments [1], with Speck achieving exceptional performance in software. Biryukov and Roy [2] analyzed Speck’s differential characteristics; Abed et al. [3] extended this analysis to Speck-128/256 specifically, finding no practical attack against the full 34-round variant with a 2^{256} key space.

The Davies-Meyer construction [4] is a foundational block-cipher-based hash mode, proven collision-resistant in the ideal cipher model. Its use here is not for collision resistance but for key derivation with a large work factor — analogous to bcrypt [5] or scrypt [6], which also use iterated operations to impose a measurable time cost on password verification.

Branchless constant-time MAC comparison follows well-established practice [7] for reducing data-dependent timing variation in the compared region, as formalized in OpenSSL’s CRYPTO_memcmp and BearSSL’s constant-time primitives.

The WS2812B LED driver employs cycle-accurate bit-banging at 8 MHz, a technique previously documented for AVR in FastLED and Adafruit NeoPixel but here re-implemented in pure assembly with analytically verified per-cycle traces.

III. System Architecture

The Ouroboros Engine is a polling-based bare-metal application with a single main loop and no interrupts, no RTOS, and no dynamic memory allocation. All buffers are statically allocated in the BSS section at link time.

III.I. Module Organization

Thirteen assembly files are assembled as a single translation unit via `.include` directives from `main.s`. This allows forward references to labels defined in later-included files — critical for shared primitives such as `speck_round_half1` — without requiring a custom linker script.

<code>main.s</code>	entry, stack init, SRAM clear, loop
<code>boot.s</code>	WDT neutralization
<code>uart.s</code>	UART: init, tx, rx, flush, prompt
<code>delay.s</code>	timers, animation, 5 s delay
<code>button.s</code>	debounce, state-machine cycling
<code>input.s</code>	UART rx, crypto pipeline trigger

<code>ouroboros.s</code>	Speck, DM hash, CTR, MAC, cipher
<code>config.s</code>	GPIO: PD3 input, PD6 output
<code>ws2812.s</code>	bit-bang LED driver, render_state
<code>dispatch.s</code>	4-opcode bytecode VM
<code>data.s</code>	flash: IV constant, cipher table
<code>variables.s</code>	BSS: all SRAM buffers

III.II. Main Loop

```
main_loop:
    RCALL delay_and_timers ; delay + timer tick
    RCALL handle_button    ; poll; fall to UART
    RJMP  main_loop        ; repeat
```

`handle_button` falls through to `handle_uart` when no press is detected, so UART is polled every iteration. The loop period is bounded by the busy-wait in `delay_and_timers`:

$$T_{\text{loop}} = \frac{4 \times 1349}{8 \times 10^6} \approx 674.5 \mu\text{s}$$

III.III. SRAM Allocation

Symbol	Bytes	Purpose
<code>round_keys</code>	272	$34 \times 8\text{B}$ Speck round keys
<code>l_buf</code>	32	Key schedule circ. buffer
<code>input_buf</code>	32	UART receive buffer
<code>hash_buf</code>	16	DM hash / CTR nonce
<code>ctr_buf</code>	16	CTR counter block
<code>result_buf</code>	48	Decrypted payload + MAC
State vars	9	mode, timers, indices
Total	425	20.8% of 2,048 B

Fig. 1: Static SRAM allocation.

IV. Cryptographic Design

IV.I. Speck-128/256

Speck-128/256 is an ARX block cipher with 128-bit blocks, 256-bit keys, and 34 rounds [1]. The encryption round function over 64-bit words $x, y \in \mathbb{Z}_{2^{64}}$ with round key k_i is:

$$x' = (\text{ROR}_8(x) + y) \bmod 2^{64} \oplus k_i$$

$$y' = \text{ROL}_3(y) \oplus x'$$

where ROR_n and ROL_n denote right and left bitwise rotation by n positions. The key schedule expands the

256-bit key, parsed as four 64-bit little-endian words (k_0, l_0, l_1, l_2) , into 34 round keys:

$$l_{i+3} = (k_i + \text{ROR}_8(l_{i \bmod 3})) \bmod 2^{64} \oplus i$$

$$k_{i+1} = \text{ROL}_3(k_i) \oplus l_{i+3}, \quad i = 0, \dots, 32$$

IV.I.I. AVR 8-bit Decomposition

Register pairs $(R_0, \dots, R_7)$ and $(R_8, \dots, R_{15})$ hold the state words x and y in little-endian byte order (LSB at lowest index).

Right rotation by 8 bits. ROR_8 of a 64-bit little-endian byte array $[b_0, b_1, \dots, b_7]$ is a cyclic left-shift of the byte array: $[b_1, b_2, \dots, b_7, b_0]$. Implemented in `speck_round_half1` as seven `MOV` register shifts plus one wrap-around move (8 cycles). The subsequent 64-bit addition uses `ADD R0, R8` and seven `ADC Ri, R(i+8)` instructions to propagate carry through the full word (8 cycles).

Left rotation by 3 bits. ROL_3 is performed in `speck_round_half2` as three sequential 1-bit left rotations over $(R_8, \dots, R_{15})$:

```
; One pass of ROL-by-1 (repeated three times):
LSL R8      ; MSB of R8 goes to Carry
ROL R9      ; carry in bit0, bit7 out
ROL R10; ...; ROL R15
ADC R8, R20 ; Carry -> bit0 of R8 (R20 = 0)
```

Three passes cost $3 \times 9 = 27$ instructions, followed by 8 EOR instructions for the XOR with $(R_0, \dots, R_7)$.

IV.I.II. Key Schedule Circular Buffer

The key schedule maintains `l_buf` (24 bytes; three 8-byte l words). The modular index $i \bmod 3$ is computed without hardware division (absent on AVR) via repeated subtraction:

```
.mod3: CPI  R16, 3
        BRLO .done
        SUBI R16, 3
        RJMP .mod3
```

The byte offset $(i \bmod 3) \times 8$ is obtained via three `LSL` shifts of the modular result.

IV.II. Davies-Meyer Key Stretching

The user passphrase p (zero-padded to 32 bytes, stored in `input_buf`) serves as the Speck-128/256 key K_u . A

Davies-Meyer hash construction [4] with deliberate key stretching is applied over a 16-byte hash buffer H :

$$H_0 = \text{IV}$$

$$H_j = E_{K_u}(H_{j-1}), \quad j = 1, 2, \dots, 24576$$

$$\hat{H} = H_{24576} \oplus \text{IV}$$

The IV is the first 128 bits of the SHA-256 initialization constants — a publicly known, non-secret value chosen to prevent the all-zero fixed point:

$$\text{IV} = 6\text{A}09\text{E}667 \text{ BB}67\text{A}E85 \text{ 3C}6\text{E}\text{F}372 \text{ A}54\text{F}\text{F}53\text{A}_{16}$$

The feed-forward XOR (the Davies-Meyer step) ensures $\hat{H}$ is not directly invertible even with E and K_u known, since recovering H_{24576} from $\hat{H} \oplus \text{IV}$ still requires retracing all 24,576 iterations. The 16-byte $\hat{H}$ is stored in `hash_buf` and used as the CTR nonce.

IV.II.I. Work Factor Analysis

Per-round cost in `encrypt_block`:

Operation	Cycles
<code>hardware_jitter_and_exe</code> (avg)	19
<code>speck_round_half1</code>	22
$8 \times (\text{LD} + \text{EOR})$ for key XOR	16
<code>speck_round_half2</code>	42
Loop control (INC, CP, BRNE)	4
Per round total	103

Fig. 2: Speck round cost breakdown.

Total per `encrypt_block` (34 rounds + load/store overhead):

$$T_{\text{block}} \approx 34 \times 103 + 100 \approx 3602 \text{ cycles} \approx 450 \mu\text{s}$$

Over 24,576 iterations of `davies_meyer_hash_loop`:

$$T_{\text{hash}} \approx 24576 \times \frac{3630}{8 \times 10^6} \approx 11.15 \text{ s}$$

This 11-second window is the hardware-enforced rate limit per authentication attempt. The data dependency $H_j = f(H_{j-1})$ prevents intra-attempt parallelism even on vector hardware.

IV.III. CTR-Mode Decryption

For block index $b \in \{0, 1, 2\}$, the 128-bit CTR block is:

$$\text{CTR}_b = \hat{H}[0 : 8] \mid b \mid 0^7$$

where $\hat{H}[0:8]$ is the 8-byte nonce, b is the 8-bit block counter (register R24), and 0^7 is 7 zero bytes. The keystream block is:

$$KS_b = E_{K_u}(CTR_b)$$

Decryption of the b -th 16-byte ciphertext block C_b from flash (LPM):

$$P_b = C_b \oplus KS_b$$

The three 16-byte plaintext blocks concatenate in `result_buf`:

- Bytes 0–15: 16-byte bytecode payload
- Bytes 16–47: 32-byte MAC field (expected: $0xAA \times 32$)

IV.IV. Branchless MAC Verification

Bytes 16–47 of `result_buf` must all equal $0xAA$ for a valid decryption. The `branchless_mac_verification` subroutine checks this in constant time.

Step 1 — OR-reduce error accumulation:

$$\delta = \bigvee_{i=16}^{47} (P[i] - 0xAA) \bmod 256$$

Each byte is checked via `SUBI R16, 0xAA`; differences accumulate into δ via `OR R17, R16`. If all bytes equal $0xAA$ then $\delta = 0$.

Step 2 — Branchless mask derivation via `NEG R17`; `SBC R17, R17`; `COM R17`:

$$\text{mask} = \begin{cases} 0xFF & \delta = 0 \quad (\text{all bytes match}) \\ 0x00 & \delta \neq 0 \quad (\text{any mismatch}) \end{cases}$$

Proof. If $\delta = 0$: `NEG(0) = 0, C = 0`; `SBC(0, 0, 0) = 0`; `COM(0) = 0xFF`. If $\delta > 0$: `NEG( $\delta$ ) = 256 -  $\delta$ , C = 1`; `SBC(256 -  $\delta$ , 256 -  $\delta$ , 1) = -1  $\equiv$  0xFF`; `COM(0xFF) = 0x00`. $\square$

Step 3 — Branchless masking:

$$P[i] \leftarrow P[i] \wedge \text{mask}, \quad \forall i \in [0, 47]$$

On failure, all 48 bytes are zeroed. The bytecode dispatcher then fetches opcode $0x00$ (END) and halts with no peripheral effect.

IV.V. Blind Cipher Selection

The flash cipher table holds N entries. All N entries are always decrypted and verified, preventing a timing oracle that reveals the matched index.

```
found ← 0xFF           // sentinel: "no match yet"
for i = 0 to N-1:
  decrypt(i) → result_buf
  verify_mac() → mask_i // 0xFF or 0x00
  r0 ← i AND mask_i     // i:pass, 0:fail
  found ← (found AND (NOT mask_i)) OR r0
```

AVR implementation (`R22 = i`, `R23 = found`, `R17 = mask_i`):

```
MOV R0, R22 ; R0 = i
AND R0, R17 ; R0 = i if pass, 0 if fail
COM R17      ; invert mask
AND R23, R17 ; R23 = old if fail, 0 if pass
OR R23, R0   ; R23 = winning index or old
```

Total decrypt+verify calls: always exactly N . If `found` $\neq 0xFF$ after the loop, that entry is re-decrypted and passed to the bytecode dispatcher.

V. Hardware Subsystems

V.I. WS2812B LED Driver

Four WS2812B LEDs on PD6 are driven by cycle-accurate bit-banging. At $f_{\text{CPU}} = 8$ MHz the clock period is 125 ns.

Bit	T_H	T_L	Period
0	2 cyc = 250 ns	8 cyc = 1000 ns	1.25 μ s
1	6 cyc = 750 ns	4 cyc = 500 ns	1.25 μ s

Fig. 3: WS2812B bit timing at 8 MHz.

`OUT PORTD, Rn` (1 cycle, single-word) is used in preference to `SBI/CBI` (2 cycles, two-word) for all GPIO transitions, ensuring deterministic single-cycle I/O. The conditional `SBRs R24, 7` (1 cycle no-skip; 2 cycles skip) selects between the 0-bit and 1-bit paths without an explicit branch instruction, keeping the critical path well-defined.

Per-bit loop trace (`send_byte`, 10 cycles per bit):

```
Cyc 1: OUT HIGH ; PD6 → 1 (always)
Cyc 2: SBRs R24, 7 ; test bit 7
+1 cyc OUT LOW ; 0-bit: T_H = 2 cyc (250 ns)
Cyc 4: LSL R24 ; shift next bit in
```

```

Cyc 5:  NOP          ; timing pad
Cyc 6:  NOP          ; timing pad
Cyc 7:  OUT LOW      ; 1-bit: T_H = 6 cyc (750 ns)
Cyc 8:  DEC R23      ; bit counter
Cyc 9-10: BRNE .bit_loop ; 2 cyc / 1 cyc

```

Interrupts are globally disabled during a fill (CLI/SEI). The WS2812B reset (>80 μ s LOW) occurs naturally between fills due to the main loop delay (674 μ s). Total fill time for 4 LEDs at 24 bits each: $4 \times 24 \times 10$ cycles = 960 cycles = 120 μ s.

V.II. UART Driver

UART0 operates at 9600 baud, 8N1. The baud rate register value:

$$\text{UBRR} = \left\lfloor \frac{8 \times 10^6}{16 \times 9600} \right\rfloor - 1 = 51$$

All UART operations are polling-based (RXC0, UDRE0 flags in UCSR0A). After each crypto pipeline completion, `uart_flush_rx` toggles RXEN0 in UCSR0B to clear the Data OverRun (DOR) flag accumulated during the 11-second hash window and flush stale bytes from the receive shift register.

V.III. Button Debounce

PD3 uses the ATmega328P internal pull-up (active-LOW). Software debounce via nested counter loop:

$$T_{\text{debounce}} = 256 \times 256 \times \frac{3}{8 \times 10^6} \approx 24.6 \text{ ms}$$

The button cycles the LED state machine in sequence: ANIM $\rightarrow$ RED $\rightarrow$ GREEN $\rightarrow$ BLUE $\rightarrow$ ANIM. Presses during STATE_INPUT are silently ignored.

V.IV. Software Timer Architecture

Two 16-bit SRAM counters are incremented once per main-loop iteration by `delay_and_timers`. No hardware timer overflow interrupts are used.

Animation advance period:

$$N_{\text{anim}} = (0x1C \ll 8) | 0xD4 = 7380$$

$$T_{\text{anim}} = 7380 \times 674.5 \mu\text{s} \approx 4.98 \text{ s}$$

Input inactivity timeout:

$$N_{\text{timeout}} = (0xAC \ll 8) | 0xF8 = 44280$$

$$T_{\text{timeout}} = 44280 \times 674.5 \mu\text{s} \approx 29.87 \text{ s}$$

Timer/Counter 0 (TCNT0) runs freely with no prescaler, providing a free-running 8-bit counter used as a hardware entropy source by `hardware_jitter_and_exe`.

VI. Bytecode Interpreter

`dispatch_program` implements a minimal, stack-less, forward-only bytecode interpreter over `result_buf`. Any unrecognized opcode halts immediately (fail-safe).

Opcode	Mnemonic	Semantics
0x00	END	Halt execution
0x01	LED_FILL	Next 3B = R, G, B; fill 4 LEDs
0x03	TX_STR	Next 1B = length; send N chars via UART
0xAA	MAC	Halt (integrity boundary marker)

Fig. 4: Bytecode instruction set.

The 0xAA opcode serves a dual role: it is the expected MAC padding value in `result_buf[16:47]`, and if any such byte reaches the dispatcher it halts safely. Because `branchless_mac_verification` zeros the entire buffer on MAC failure, the dispatcher always encounters 0x00 (END) on invalid input.

Reference 16-byte payload for the mapping "hello" $\rightarrow$ "world":

```

01 FF 00 FF          ; LED_FILL(255,0,255) purple
03 07                ; TX_STR(len=7)
77 6F 72 6C 64 0D 0A ; "world\r\n"
00                  ; END
00 00                ; zero padding to 16 bytes

```

VII. Side-Channel Countermeasures

VII.I. Hardware Timing Jitter

`hardware_jitter_and_exe` is invoked before every `encrypt_block` call:

```

IN    R16, 0x26 ; read TCNT0 (free-running)
ANDI  R16, 0x07 ; 3-bit mask  $\rightarrow$  jitter [0, 7]
.jit:
DEC   R16
BRPL .jit      ; loop while  $\geq 0$  (signed)

```

The bottom 3 bits of TCNT0 provide jitter of 0–7 cycles (0–875 ns) per encrypt call. Over 24,576 calls per authentication attempt, this changes trace alignment characteristics. This paper does not provide a measured DPA/FI success-rate dataset.

VII.II. Constant-Time MAC Comparison

`branchless_mac_verification` processes all 32 MAC bytes unconditionally before computing the mask. No branch depends on any individual byte value. Within that routine, execution structure does not depend on the compared MAC bytes.

VII.III. Constant Cipher Iteration Count

`blind_xor_decryption` and `branchless_mac_verification` are called exactly CIPHER_ENTRIES times regardless of early matches. The winning index is selected via bitwise operations with no early exit. This removes that specific match-position timing difference.

VII.IV. Key Material Zeroization

`clear_input_buf` writes 32 zero bytes to `input_buf` immediately after each authentication attempt. The 272-byte `round_keys` and 16-byte `hash_buf` are overwritten at the start of the next attempt.

VIII. Security Analysis

VIII.I. Threat Model

The adversary has: (1) unrestricted physical access; (2) full observation of UART traffic; (3) high-resolution power side-channel measurement capability; (4) a complete copy of the firmware binary; and (5) arbitrary offline compute resources. The single secret is the passphrase, present in SRAM only during the authentication window.

VIII.II. Attack Analysis

Online brute-force. Each attempt requires 11 s on the ATmega328P, limiting throughput to 3 attempts/minute via physical interaction alone. For a 6-character lowercase passphrase ($26^6 \approx 3 \times 10^8$ candidates):

$$T_{\text{online}} \approx \frac{3 \times 10^8}{3 \text{ per minute}} \approx 190 \text{ yr}$$

Offline simulation. A GPU capable of 10^{12} Speck-128/256 operations/s must still perform 24,576 sequential encryptions per candidate (due to the chain dependency $H_j = f(H_{j-1})$). Per-candidate GPU cost: $\frac{24576}{10^{12}} \approx 24.6 \text{ ns}$.

For the strict adversarial scenario used in this paper, assume an externally enforced 32-character non-dictionary random master key over lowercase + uppercase + digits ($|\Sigma| = 62$, $n = 32$):

$$T_{\text{offline}} = 62^{32} \times 24.6 \text{ ns} \approx 5.59 \times 10^{49} \text{ s} \approx 1.77 \times 10^{42} \text{ yr}$$

Using an optimistic Grover-style average-time toy model (favorable to attacker):

$$T_q \approx \frac{\sqrt{62^{32}}}{2} \times 24.6 \text{ ns} \approx 5.86 \times 10^{20} \text{ s} \approx 1.86 \times 10^{13} \text{ yr}$$

These are model outputs under explicit assumptions, not measured throughput benchmarks or proofs of post-quantum security.

Flash readback. Lock bits 0x00 disable all external ISP read modes. Recovery requires a chip-erase, destroying all flash contents including the cipher table.

Ciphertext malleability. Any modification to a cipher table entry produces a different decrypted value. The probability of an arbitrary modification passing the 32-byte 0xAA MAC by chance is $2^{-256} \approx 0$.

Power analysis. Hardware jitter of 0–7 cycles per encrypt call misaligns power traces. Combined with the 24,576-call depth, this increases trace alignment cost. This paper does not provide a measured DPA/FI success-rate dataset.

Cold-boot attack. `input_buf` is zeroed immediately after use by `clear_input_buf`, minimizing the window during which key material is resident in SRAM.

VIII.III. Limitations

The CTR nonce $\hat{H}[0 : 8]$ is reused across all N cipher table entries within a single authentication session (differing only in block counter b). Two different passwords producing the same $\hat{H}$ (a 64-bit preimage collision) would share a keystream — with probability $\approx 2^{-64}$ per pair, negligible in practice. No practical attack on full Speck-128/256 has been published [2][3].

The 32-character random base62 key assumption is an analysis assumption, not a firmware-enforced input policy. The firmware accepts up to 32 bytes and pads shorter input; real-world security depends on operator policy and secret entropy.

IX. Implementation Metrics

Metric	Value
Target MCU	ATmega328P (8-bit AVR)
Clock	8 MHz, internal RC oscillator
SRAM used	425 / 2,048 B (20.8%)
Cipher primitive	Speck-128/256, 34 rounds
Key size / Block size	256 bit / 128 bit
Hash iterations	24,576 per attempt
Auth time (hash only)	≈ 11.15 s per attempt
UART baud rate	9600 baud, 8N1
LEDs	$4 \times$ WS2812B
Jitter entropy	3 bits (TCNT0[2:0])
Input timeout	≈ 30 s inactivity
Max passphrase	32 bytes
Source lines	≈ 550 (incl. comments)
External deps	None
ISRs used	None

Fig. 5: Implementation summary.

X. Conclusion

The Ouroboros Engine demonstrates that a complete cryptographic authentication system — incorporating a standard block cipher, key-stretching hash construction, CTR-mode decryption, constant-time MAC verification, branchless cipher selection, and a bytecode interpreter — can be implemented entirely in pure assembly on an 8-bit microcontroller with no C runtime support, no interrupt handlers, and no external library dependencies.

The principal engineering contributions are:

1) Byte-level Speck-128/256 on 8-bit AVR. A complete decomposition of 64-bit ARX operations into register rotation sequences and multi-precision carry

chains, analytically verified against a Python reference implementation.

2) Davies-Meyer key stretching with 24,576 iterations. A hardware-bound work factor of approximately 11 seconds per attempt on the target clock, increasing modeled brute-force cost under stated assumptions.

3) Branchless constant-time MAC verification. A 2’s-complement masking technique using NEG, SBC, and COM that achieves data-independent execution time with no multiply or divide instructions.

4) Hardware-timer-derived timing jitter. Exploitation of the free-running Timer/Counter 0 as a zero-cost timing-variation source, requiring no additional hardware beyond the standard ATmega328P peripherals.

5) Minimal 4-opcode bytecode interpreter. A stack-less, jump-free VM that allows the authenticated payload to dynamically control peripherals without hardcoding output behavior into the firmware — new challenge-response mappings are added by re-encrypting the cipher table without modifying any assembly.

The design prioritizes auditability: every algorithm is a published, peer-reviewed primitive; key timing thresholds are analytically derived with explicit cycle counts; and the entire codebase fits in 550 lines of annotated assembly. The core implementation behavior and estimator assumptions are directly inspectable, with no hidden compiler transformations or opaque runtime layers.

Future work includes EEPROM-backed persistent challenge-response pairs, extending the bytecode to support conditional logic, and replacing TCNT0 as the jitter source with true hardware noise (e.g., ADC noise floor sampling) for different timing-variation characteristics.

XI. References

- [1] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, “The SIMON and SPECK families of lightweight block ciphers,” *Cryptology ePrint Archive*, Rep. 2013/404, 2013. [Online]. Available: <https://eprint.iacr.org/2013/404>
- [2] A. Biryukov and A. Roy, “Differential analysis of block ciphers SIMON and SPECK,” in *Fast Software Encryption (FSE 2014)*, Lecture Notes in Computer Science, vol. 8540, Springer, 2015, pp. 546–570.
- [3] F. Abed, E. List, S. Lucks, and J. Wenzel, “Differential cryptanalysis of round-reduced Simon and Speck,” in *Fast*

- Software Encryption (FSE 2014)*, Lecture Notes in Computer Science, vol. 8540, Springer, 2015, pp. 525–545.
- [4] B. Preneel, R. Govaerts, and J. Vandewalle, “Hash functions based on block ciphers: A synthetic approach,” in *Advances in Cryptology — CRYPTO ‘93*, Lecture Notes in Computer Science, vol. 773, Springer, 1994, pp. 368–378.
- [5] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proc. USENIX Annual Technical Conf. (USENIX ATC ‘99)*, Monterey, CA, 1999, pp. 81–91.
- [6] C. Percival and S. Josefsson, “The scrypt Password-Based Key Derivation Function,” RFC 7914, Internet Engineering Task Force, Aug. 2016.
- [7] D. J. Bernstein, “Cache-timing attacks on AES,” Dep. Mathematics, Statistics, and Computer Science, Univ. of Illinois at Chicago, Tech. Rep., 2005. [Online]. Available: <https://cr.yp.to/antiforgery/cachetiming-20050414.pdf>
- [8] Atmel Corp., *ATmega328P 8-bit AVR Microcontroller Datasheet*, Rev. 8161D, 2015.
- [9] R. Beaulieu et al., “SIMON and SPECK: Block Ciphers for the Internet of Things,” Cryptology ePrint Archive, Rep. 2015/585, 2015.
- [10] WorldSemi, *WS2812B Intelligent Control LED Integrated Light Source Datasheet*, Rev. 1.0, 2014.